

Perfect. We'll continue exactly like a **Principal Engineer's Architecture Handbook**. I'll include the architecture, the actual code, the libraries, and explain **what happens internally**.

Company Brain Engineering Handbook

Chapter 1 – Enterprise Architecture (Part 2A)

Extraction Worker → Module 2 Handoff → Chunk Worker

10. Extraction Worker

What is an Extraction Worker?

Many people think Celery itself extracts PDFs.

No.

Celery only executes **your Python code**.

The worker is simply a long-running Python process.

Imagine you start it using:

Bash

```
celery -A app.celery_app worker \  
  --loglevel=info \  
  --queues=extract_queue \  
  --concurrency=4
```

Let's understand this.

```
celery
```

Starts the Celery program.

```
-A app.celery_app
```

Loads your Celery application.

Suppose your project is

```
company-brain/
```

```
|
|
├─ app/
|
|   └─ celery_app.py
|
|   └─ workers/
|
|   └─ api/
|
|   └─ ...
```

Celery imports

Python

Run

```
app.celery_app
```

`worker`

Starts a worker process.

Without this,

nothing executes.

`--queues=extract_queue`

This worker only listens to

`extract_queue`

Later we will have

```
extract_queue

chunk_queue

embedding_queue

index_queue

notification_queue
```

Each worker only listens to its queue.

```
--concurrency=4
```

This worker can process

```
4 documents  
  
simultaneously.
```

How does Celery know which function to execute?

Suppose

Python

Run

```
extract_document.delay(document_id=101)
```

is called.

Celery stores

JSON

```
{  
  "task":  
    "app.workers.tasks.extract_document",  
  
  "args": [101]  
}
```

Worker receives

Task Name

↓

```
app.workers.tasks.extract_document
```

Worker imports

Python

Run

```
from app.workers.tasks  
import extract_document
```

Then executes

Python

Run

```
extract_document(101)
```

Nothing magical.

It simply imports your function.

Folder Structure

I recommend

```
company-brain/  
  
|  
├─ app/  
  
|  
├─ api/  
|   documents.py  
  
|  
├─ workers/  
|   tasks.py  
  
|  
├─ services/  
|   extraction/  
|  
|   chunking/  
|  
|   embeddings/  
|  
└─ models/
```

```
|
|
├─ db/
|
|
├─ utils/
|
|
└─ celery_app.py
```

Notice

Business Logic

is NOT

inside

tasks.py

tasks.py

only starts services.

celery_app.py

Python

Run

```
from celery import Celery

celery = Celery(

    "company_brain",

    broker="redis://localhost:6379/0",

    backend="redis://localhost:6379/1"

)

celery.conf.update(

    task_serializer="json",

    result_serializer="json",

    accept_content=["json"],
```

```
task_track_started=True,  
  
task_time_limit=1800,  
  
)
```

Let's explain.

Why broker?

Python

Run

```
broker="redis://..."
```

Broker

=

Message Queue

Redis stores

tasks.

Why backend?

Celery can optionally store task results.

Example

Task

↓

Completed

or

Failed

The backend stores this state.

Worker Task

Now

Python

Run

```
from app.celery_app import celery

from app.services.extraction.pipeline import ExtractionPipeline

@celery.task(

    bind=True,

    autoretry_for=(Exception,),

    retry_backoff=True,

    retry_kwargs={"max_retries":5}

)

def extract_document(

    self,

    document_id:int

):

    pipeline = ExtractionPipeline()

    pipeline.run(document_id)
```

Let's explain everything.

bind=True

Normally

Python

Run

```
def task():
```

has no task object.

With

Python

Run

```
bind=True
```

Celery passes

Python

Run

```
self
```

Now

you can access

Python

Run

```
self.request.id

self.retry()

self.request.retries
```

Useful

for logging

and retries.

autoretry_for

Suppose

AWS

is temporarily unavailable.

Without retry

↓

Task fails forever.

With

Python

Run

```
autoretry_for=(Exception,)
```

Celery automatically retries.

retry_backoff=True

Without

backoff

Retries

1 sec

1 sec

1 sec

Bad.

With

backoff

1 sec

2 sec

4 sec

8 sec

16 sec

Much better.

ExtractionPipeline

Notice

Worker

doesn't contain extraction logic.

Instead

Python

Run

```
pipeline = ExtractionPipeline()
```

Good architecture.

Why?

Suppose

tomorrow

you create

REST API

that also extracts PDFs.

It can reuse

ExtractionPipeline.

Business logic

stays separate.

ExtractionPipeline

Python

Run

```
class ExtractionPipeline:

    def run(

        self,

        document_id

    ):

        document = self.load_document(

            document_id

        )
```



```
document_id
```

```
)
```

Let's understand

every function.

load_document()

Reads PostgreSQL.

SQLAlchemy

Python

Run

```
document = (  
  
    session.query(Document)  
  
    .filter(  
  
        Document.id==document_id  
  
    )  
  
    .one()  
  
)
```

Generated SQL

SQL

```
SELECT *  
  
FROM documents  
  
WHERE id=101;
```

Result

```
id  
  
101
```

filename

Financial_Report.pdf

s3_path

tenant55/docs/101.pdf

status

uploaded

download_pdf()

Uses

Python

Run

boto3

AWS SDK

Example

Python

Run

```
response = s3.get_object(  
  
    Bucket="companybrain",  
  
    Key=document.s3_path  
  
)
```

Question

File is encrypted.

How can worker read?

Answer

S3 automatically

decrypts it

because

Worker

has IAM permission.

Worker never sees

encrypted bytes.

AWS handles

everything.

extract_pages()

Uses

Python

Run

PyMuPDF

Library

Python

Run

```
import fitz
```

Example

Python

Run

```
pdf = fitz.open(stream=pdf_bytes)

pages = [

    page

    for page in pdf

]
```

Now

every page

is an object.

extract_text()

Library

PyMuPDF

or

pdfplumber

Example

Python

Run

```
text = page.get_text()
```

OCR

Suppose

page

contains only image.

PyMuPDF returns

nothing.

Then

Python

Run

```
pytesseract
```

runs.

Architecture

PDF

↓

Image

↓

OCR

↓

Text

extract_tables()

Libraries

Camelot

pdfplumber

Tabula

depending

on table type.

extract_images()

PyMuPDF

can extract

embedded images.

Example

Python

Run

```
page.get_images()
```

extract_metadata()

Metadata

includes

Author

Title

Subject

Keywords

Producer

Creation Date

Modification Date

Again

PyMuPDF

can read this.

store_results()

Now

everything

goes

to PostgreSQL.

Tables

documents

pages

sections

tables

images

metadata

Notice

Redis

stores

nothing.

Redis

only

stored

the task.

Database Example

documents

id	status
101	extracting

↓

After extraction

id	status
101	extracted

Now

Module 1

is complete.

What next?

Should

Worker

call

Chunker

directly?

No.

That creates

tight coupling.

Instead

Worker publishes

another task.

Python

Run

```
chunk_document.delay(  
  
    document_id  
  
)
```

Redis

receives

```
chunk_queue  
  
↓  
  
chunk_document  
  
↓  
  
document_id=101
```

Extraction Worker

is done.

Why Queue the Next Task Instead of Calling It Directly?

Imagine the extraction worker directly called the chunking function.

Python

Run

```
extract_document()  
  
↓
```

```
chunk_document()
```

↓

```
embed_document()
```

↓

```
index_document()
```

If embedding fails after 20 minutes:

- the whole pipeline fails,
- retries become difficult,
- extraction might run again unnecessarily.

Instead, each stage finishes, saves its output, and queues the next stage.

That gives you:

- Independent retries.
- Independent scaling.
- Better monitoring.
- Clear progress tracking.
- Fault isolation.

Module 2 Starts

The Chunk Worker is running separately.

Bash

```
celery -A app.celery_app worker \  
  --queues=chunk_queue \  
  --concurrency=8
```

It constantly asks Redis:

```
Do you have any chunking work?
```

Redis replies:

```
chunk_document(document_id=101)
```

The worker imports:

Python

Run

```
from app.workers.tasks import chunk_document
```

and executes it.

Chunk Worker

Python

Run

```
@celery.task(
    bind=True,
    autoretry_for=(Exception,),
    retry_backoff=True
)
def chunk_document(self, document_id: int):

    from app.services.chunking.pipeline import ChunkingPipeline

    pipeline = ChunkingPipeline()

    pipeline.run(document_id)
```

Notice something important.

The chunk worker **does not**:

- Download the PDF again.
- Run OCR again.
- Extract tables again.

Instead, it loads the structured output already stored by Module 1.

For example:

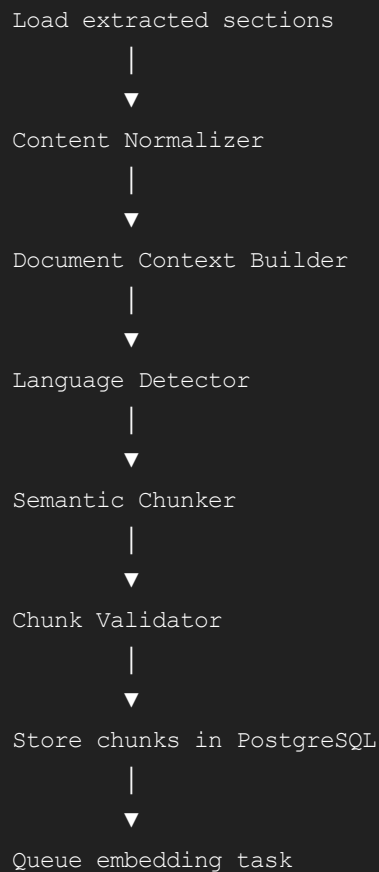
Python

Run

```
sections = (
    session.query(Section)
    .filter(Section.document_id == document_id)
    .all()
)
```

This is much faster than reprocessing the original file.

Chunking Pipeline (High Level)



At the end:

Python

Run

```
embed_document.delay(document_id)
```

And the pipeline continues.

End of Part 2A

In **Part 2B**, we'll go even deeper into:

- How Redis acknowledges tasks (ACK/NACK).
- What happens if a worker crashes.
- Embedding Worker implementation.
- `pgvector` storage and HNSW indexes.
- Vector database schema.
- Retrieval pipeline.
- Failure recovery.
- Production monitoring.
- Scaling to millions of documents.

From this point onward, we'll essentially be building the actual Company Brain architecture, component by component, with production-level code and explanations.